

Proseminar: Sortieralgorithmen

Maya Neudorf

18. Mai 2026

Inhaltsverzeichnis

- ▶ Definition Ordnung
- ▶ das allgemeine Sortierproblem
- ▶ Sortieren durch sukzessive Auswahl
- ▶ Mergesort
- ▶ zentraler Satz zur Anzahl der Vergleiche
- ▶ Quicksort
- ▶ Heapsort

Ordnung Definition

Partielle Ordnung:

Sei S eine Menge. Eine Relation $R \subseteq S \times S$, wenn folgende Eigenschaften gelten:

- (a) $(a, a) \in R$ (Reflexivität);
- (b) $((a, b) \in R \wedge (b, a) \in R) \Rightarrow a = b$ (Antisymmetrie)
- (c) $((a, b) \in R \wedge (b, c) \in R \Rightarrow (a, c) \in R$ (Transitivität)

Totale Ordnung:

Eine partielle Ordnung R von S , wenn $(a, b) \in R$ oder $(b, a) \in R$ für alle $a, b \in S$ gilt

Das allgemeine Sortierproblem

Input

eine endliche Menge S mit einer partiellen Ordnung $\preceq$

Ausgabe

Berechne eine Bijektion $f: \{1, \dots, n\} \rightarrow S$ mit $f(j) \not\preceq f(i)$ für alle $1 \leq i < j \leq n$ (eine sortierte Liste)

naives Verfahren

Alle $n!$ Permutationen ausprobieren und testen ob für alle $n(n-1)/2$ Paare (i, j) $f(j) \not\preceq f(i)$ gilt.

Sortieren durch sukzessive Auswahl

Eingabe:

eine Menge $S = \{s_1, \dots, s_n\}$; eine partielle Ordnung $\preceq$ auf S

Ausgabe:

eine Bijektion $f : \{1, \dots, n\} \rightarrow S$ mit $f(j) \not\preceq f(i)$ für alle $1 \leq i < j \leq n$.

for $i \leftarrow 1$ **to** n **do** $f(i) \leftarrow s_i$

for $i \leftarrow 1$ **to** n

for $j \leftarrow i$ **to** n

if $f(j) \preceq f(i)$ **then** **swap**($f(i), f(j)$)

output f

Beweis für Korrektheit des Algorithmus

Wir zeigen, dass für alle $1 \leq i \leq j \leq n$ nach Iteration (i,j) folgendes gilt:

- (a) $f(k) \not\leq f(i)$ für alle $1 \leq h < i$ und $h < k \leq n$ (vor i ist die Liste sortiert)
- (b) $f(k) \not\leq f(i)$ für alle $i < k \leq j$ (i ist kleiner als alles rechts davon)

Für b) 1. Fall $f(j) \not\leq f(i)$: es wird nichts getauscht. b) gilt also

2. Fall: $f(j) \leq f(i)$, betrachte Index k mit $i < k \leq j$: für $j = k$ gilt wegen der Antisymmetrie b) nach dem swap-Befehl. für $k < j$ galt zuvor $f(k) \not\leq f(i)$ und somit $f(k) \not\leq f(j)$. Es gilt also b).

Die Laufzeit des Algorithmus ist $O(n^2)$.

Sortieren nach Schlüsseln

Input

eine endliche Menge S , sowie eine Funktion $k : S \rightarrow K$ und eine totale Ordnung auf K .

Ausgabe

Berechne eine Bijektion $f : \{1, \dots, n\} \rightarrow S$ mit $k(j) \not\leq k(i)$ für alle $1 \leq i < j \leq n$ (eine sortierte Liste von Schlüsseln)

Beispiele

Bucketsort, Insertion sort, Bubblesort

Mergesort

Input

eine Menge $S = \{s_1; \dots; s_n\}$; eine durch Schlüssel induzierte partielle Ordnung $\preceq$ auf S .

Ausgabe

Berechne eine Bijektion $f: \{1, \dots, n\} \rightarrow S$ mit $f(j) \not\preceq f(i)$ für alle $1 \leq i < j \leq n$. **if** $|S| > 1$ **then**

- ▶ sei $S = S_1 \dot{\cup} S_2$ mit $|S_1| = \lfloor \frac{n}{2} \rfloor$ und $|S_2| = \lceil \frac{n}{2} \rceil$
- ▶ sortiere S_1 und S_2 (rekursiv mit Mergesort)
- ▶ Verschmelze die Sortierungen von S_1 und S_2 zu Sortierung S .

Mergesort sortiert n Elemente in der Laufzeit $O(n \log n)$

Beweis:

Sei $c \in \mathbb{N}$ mit $T(1) \leq c$ und

$$T(n) \leq T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + cn \quad \text{für alle } n \geq 2.$$

Behauptung: $T(2^k) \leq c(k+1)2^k$ für alle $k \in \mathbb{N} \cup \{0\}$.

Induktion über k : Für $k = 0$ trivial.

Für $k \in \mathbb{N}$ gilt:

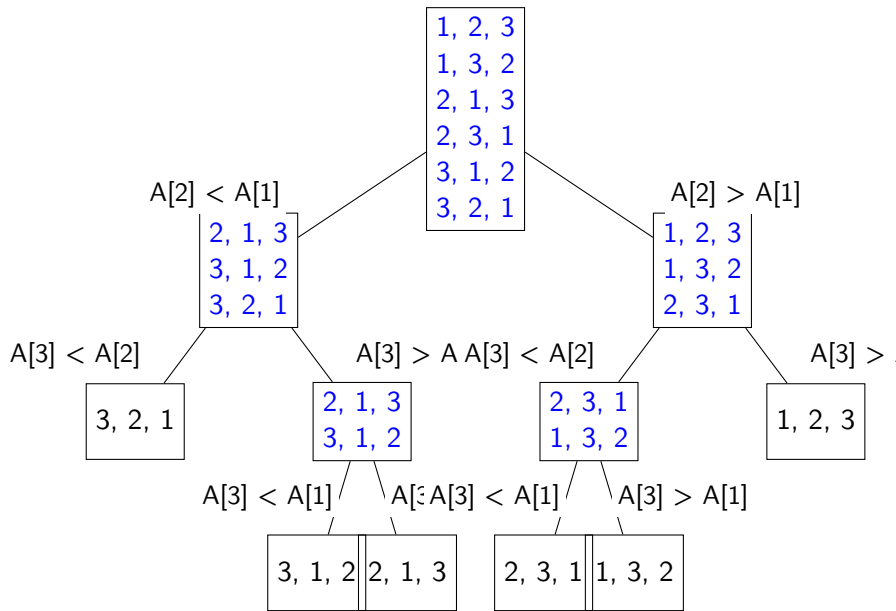
$$T(2^k) \leq 2 \cdot T(2^{k-1}) + c2^k \leq 2 \cdot c k 2^{k-1} + c2^k = c(k+1)2^k.$$

$\Rightarrow n \in \mathbb{N}$ mit $k := \lceil \log_2 n \rceil < 1 + \log_2 n$:

$$T(n) \leq T(2^k) \leq c(k+1)2^k < 2c(2 + \log_2 n)n = O(n \log n).$$

Satz

Jeder Sortieralgorithmus, der nur mit paarweisen Vergleichen von zwei Elementen arbeitet, benötigt zum sortieren einer n -elementigen Menge mindestens $\log_2(n!)$ Vergleiche.



Beweis:

Gegeben: Eingabemenge $S = \{1, \dots, n\}$ mit einer totalen Ordnung

Gesucht: einer der $n!$ Permutationen (Blätter im Vergleichsbaum).

Jeder Vergleich halbiert die Anzahl an möglichen Permtationen (ein Vergleich ist ein Schritt im Baum). Ein Algorithmus braucht also k Vergleiche mit $2^k \geq n! \Rightarrow k \geq \log_2(n!)$.

Im Allgemeinen werden mehr Vegleiche gebraucht(Beispiel $n=5$)

Quicksort

Input

eine Menge $S = \{s_1; \dots; s_n\}$; eine durch Schlüssel induzierte partielle Ordnung $\preceq$ auf S .

Ausgabe

Berechne eine Bijektion $f: \{1, \dots, n\} \rightarrow S$ mit $f(j) \not\preceq f(i)$ für alle $1 \leq i < j \leq n$.

if $|S| > 1$ **then**

- ▶ wähle $x \in S$ beliebig
- ▶ $S_1 := \{s \in S \mid s \preceq x\} \setminus \{x\}$
- ▶ $S_2 := \{s \in S \mid x \preceq s\} \setminus \{x\}$
- ▶ $S_x := S \setminus (S_1 \cup S_2)$
- ▶ Sortiere S_1 und S_2 (soweit nicht leer) rekursiv mit Quicksort
- ▶ Füge die sortierten Mengen S_1 , S_x und S_2 aneinander

Quicksort sortiert n Elemente in der Laufzeit $O(n^2)$

Beweis

Es existiert ein $c \in \mathbb{N}$ mit $T(1) \leq c$.

$T(n) \leq cn + \max_{0 \leq l \leq n-1} (T(l) + T(n-1-l))$ für alle $n \geq 2$.

Behauptung: $T(n) \leq cn^2$

Beweis per Induktion über n : Für $n = 1$ trivial.

$T(n) \leq \max_{0 \leq l \leq n-1} (cl^2 + c(n-1-l)^2) = cn + c(n-1)^2 < cn^2$

Vorteil: es wird kein zusätzlicher temporärer Speicherplatz benötigt

die Wahl des x

x als erstes Element oder zufällig

unkompliziert, für bereits sortierte Listen oder kurze Listen
gut(Laufzeit: $O(n^2)$)

x als Median

Median finden ist relativ aufwendig, leichteres sortieren, gut für
lange Listen(Laufzeit: $O(n)$ (Median ermitteln)+ $O(n\log n)$
(sortieren))

Der Erwartungswert der Laufzeit von Random-Quicksort für n Elemente ist $O(n \log(n))$

Beweis

Es existiert ein $c \in \mathbb{N}$ mit $\overline{T}(1) \leq c$.

$$\overline{T}(n) \leq cn + \frac{1}{n} \sum_{i=1}^n (\overline{T}(i-1) + \overline{T}(n-i)) = cn + \frac{2}{n} \sum_{i=1}^{n-1} \overline{T}(i)$$

für alle $n \geq 2$. **Behauptung:** $\overline{T}(n) < 2cn(1 + \ln n)$.

Wir zeigen dies per Induktion über n . Für $n = 1$ ist die Aussage trivial. Für $n \geq 2$ erhalten wir mit der Induktionsvoraussetzung:

$$\begin{aligned} \overline{T}(n) &\leq cn + \frac{2}{n} \sum_{i=1}^{n-1} 2ci(1 + \ln i) \\ &\leq cn + \frac{2c}{n} \int_1^n 2x(1 + \ln x) dx \\ &= cn + \frac{2c}{n} \left[x^2(1 + \ln x) - \frac{x^2}{2} \right]_1^n \\ &= cn + 2cn(1 + \ln n) - cn - \frac{c}{n} < 2cn(1 + \ln n) \end{aligned}$$

Heapsort

Definition Heap

Ein **Heap** für eine endliche Menge S mit Schlüsseln $k : S \rightarrow K$ und einer totalen Ordnung $\leq$ von K ist eine Arboreszenz A mit einer Bijektion $f : V(A) \rightarrow S$, so dass die Heapordnung $k(f(v)) \leq k(f(w))$ für alle $(v, w) \in E(A)$ gilt.

(Ein Baum, in dem jedes Elternteil einen kleineren Wert hat als seine Kinder)

Heapsort sortiert n Elemente in der Laufzeit $O(n \log(n))$

Der Baum ist maximal $\log_2(n)$ tief $\Rightarrow$ eine Operation kostet $O(\log(n))$ und es müssen n Elemente hinzugefügt werden.

Literaturverzeichnis

- ▶ Stefan Hougary and Jens Vygen: Algorithmische Mathematik (2018, Springer Spektrum)
- ▶ Folien zu Algorithmen und Datenstrukturen von Prof.Dr. Markus Nebel(Sose2024)